Nischal Shrestha
Coventry University Student ID Number: 17103446
B.Sc. (Hons.) Computing, Softwarica College of IT and E-commerce, Coventry University ST4005CEM Database System
Giriraj Rawat

Question 1: Create Tables

1. **Publishers** (publisher_id PK, publisher_name, city, established_year)

```
create table publishers (
publisher_id int primary key ,
publisher_name varchar(30),
city varchar(30),
established_year year
);
```

2. **Authors** (author_id PK, author_name, country, birth_year)

```
create table authors(
author_id int primary key,
author_name varchar(30),
country varchar(30),
birth_year year
);
```

3. **Books** (book_id PK, title, publication_year, pages, publisher_id FK → Publishers, author_id FK → Authors) Use ON DELETE CASCADE for both FKs.

```
create table books (
book_id int primary key,
title varchar(50),
publication_year year,
pages int,
publisher_id int,
author_id int,
foreign key (publisher_id) references publishers(publisher_id) on delete cascade,
foreign key (author_id) references authors(author_id) on delete cascade
);
```

4. **Members** (member_id PK, full_name, join_date (DATE), membership_type for example 'Basic', 'Premium', phone)

```
34 • create table members (
35   member_id int primary key ,
36   full_name varchar(30),
37   join_date date default (curdate()),
38   membership_type enum("Basic","Premium","Phone")
39   );
```

5. **Borrowing** (borrow_id PK, book_id FK **From Table** Books, member_id FK **From Table** Members, borrow_date (DATE), due_date (DATE), return_date (DATE = can be NULL if not yet returned)) Use ON DELETE CASCADE for both FKs.

```
• create table borrowing (
borrow_id int primary key auto_increment,
book_id int,
member_id int,
borrow_date date default (curdate()),
due_date date,
return_date date,
foreign key (book_id) references books(book_id) on delete cascade,
foreign key (member_id) references members(member_id) on delete cascade
);
```

6. **Fines** (fine_id PK, borrow_id FK → Borrowing (UNIQUE = one fine per borrowing record if applicable), fine_amount (DECIMAL), paid_status — BOOLEAN or 'Paid'/'Unpaid', paid_date (DATE = NULL if unpaid)) Use ON DELETE CASCADE for the FK.

```
61 • create table fines(
62     fine_id int primary key auto_increment,
63     borrow_id int unique,
64     fine_amount decimal(10,2),
65     paid_status enum("Paid","Unpaid"),
66     paid_date date default null,
67     foreign key (borrow_id) references borrowing(borrow_id) on delete cascade
68     );
```

Question 2: Insert Sample Data

```
INSERT INTO publishers VALUES
(1, 'Penguin Books', 'London', 1935),
(2, 'HarperCollins', 'New York', 1989),
(3, 'Oxford Press', 'Oxford', 2006),
(4, 'Macmillan', 'London', 1943),
(5, 'Scholastic', 'New York', 1920),
(6, 'Bloomsbury', 'London', 1986),
(7, 'Random House', 'New York', 1927),
(8, 'Wiley', 'New Jersey', 1907),
(9, 'Pearson', 'London', 1944),
(10, 'McGraw Hill', 'New York', 1988),
(11, 'Cengage', 'Boston', 1904),
(12, 'MIT Press', 'Cambridge', 1962),
(13, 'Routledge', 'London', 2024),
(14, 'Springer', 'Berlin', 2001),
(15, 'O Reilly', 'California', 1978);
```

```sql
INSERT INTO authors VALUES
(1,  'Laxmi Prasad Devkota', 'Nepal', 1909),
(2,  'Bishweshwar Prasad Koirala', 'Nepal', 1914),
(3,  'Lil Bahadur Chhetri',  'Nepal', 1933),
(4,  'Narayan Wagle',        'Nepal', 1971),
(5,  'Buddhisagar Chapain',  'Nepal', 1971),
(6,  'Nayan Raj Pandey',     'Nepal', 1972),
(7,  'Indra Bahadur Rai',    'Nepal', 1927),
(8,  'Parijat',              'Nepal', 1937),
(9,  'Guru Prasad Mainali',  'Nepal', 1907),
(10, 'Bhupi Sherchan',       'Nepal', 1936),
(11, 'Moti Ram Bhatta', 'Nepal', 1901);
```

```sql
INSERT INTO members VALUES
(1,  'Nischal Shrestha',  '2022-01-10', 'Premium', '9841000001'),
(2,  'Nirjal Shrestha',      '2022-03-15', 'Basic',   '9841000002'),
(3,  'Priya Poudel',      '2022-05-20', 'Premium', '9841000003'),
(4,  'Rohan Karki',       '2023-01-05', 'Basic',   '9841000004'),
(5,  'Sita Thapa',        '2023-02-14', 'Premium', '9841000005'),
(6,  'Bikash Rai',        '2023-04-18', 'Basic',   '9841000006'),
(7,  'Anita Gurung',      '2023-06-22', 'Premium', '9841000007'),
(8,  'Dipesh Magar',      '2023-08-30', 'Basic',   '9841000008'),
(9,  'Kamala Tamang',     '2024-01-11', 'Basic',   '9841000009'),
(10, 'Suresh Limbu',      '2024-02-25', 'Premium', '9841000010'),
(11, 'Bina Maharjan',     '2024-03-03', 'Basic',   '9841000011'),
(12, 'Prakash Shrestha',  '2024-04-17', 'Premium', '9841000012');
```

```sql
INSERT INTO books VALUES
(1,  'Muna Madan',          1936, 108, 1,  1),
(2,  'Sumnima',             1971, 224, 1,  1),
(3,  'Paralu',              1966, 186, 2,  2),
(4,  'Seto Dharti',         1974, 312, 2,  2),
(5,  'Basain',              1957, 142, 3,  3),
(6,  'Agni ko Katha',       1962, 198, 3,  3),
(7,  'Palpasa Cafe',        2005, 280, 4,  4),
(8,  'Nabheko Manche',      2009, 264, 4,  4),
(9,  'Karnali Blues',       2008, 296, 5,  5),
(10, 'Mayur Timse',         2015, 210, 6,  6),
(11, 'Alikhit',             2018, 188, 7,  7),
(12, 'Pharkera Herda',      2010, 176, 8,  8),
(13, 'Swasni Manchhe',      1967, 152, 9,  9);
```

```sql
INSERT INTO borrowing VALUES
(1,  1,  1,  '2024-01-01', '2024-01-15', '2024-01-10'),
(2,  3,  2,  '2024-01-05', '2024-01-19', '2024-01-14'),
(3,  5,  3,  '2024-02-01', '2024-02-15', '2024-02-10'),
(4,  7,  4,  '2024-02-10', '2024-02-24', '2024-02-20'),
(5,  2,  1,  '2024-03-01', '2024-03-15', '2024-03-28'),
(6,  4,  2,  '2024-03-10', '2024-03-24', '2024-04-06'),
(7,  6,  3,  '2024-04-01', '2024-04-15', '2024-04-30'),
(8,  8,  5,  '2024-04-10', '2024-04-24', '2024-05-05'),
(9,  9,  6,  '2024-05-01', '2024-05-15', '2024-05-27'),
(10, 10, 7,  '2025-01-01', '2025-01-15', NULL),
(11, 12, 8,  '2025-01-10', '2025-01-24', NULL),
(12, 13, 9,  '2025-02-01', '2025-02-15', NULL),
(13, 1,  10, '2025-02-10', '2025-02-24', NULL);
```

```
INSERT INTO fines VALUES
(1, 5, 130.00, "unpaid", NULL),
(2, 6, 220.00, "paid",  '2024-05-10'),
(3, 7, 300.00, "unpaid", NULL),
(4, 8, 105.00, "paid",  '2024-06-01'),
(5, 9, 240.00, "unpaid", NULL);
```

Question 3: SQL Queries

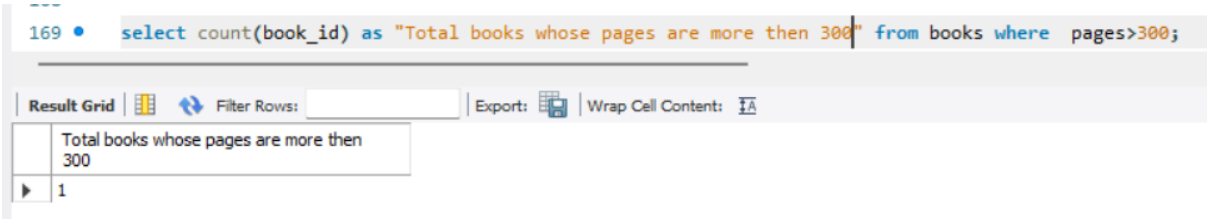
Write separate SQL queries for each task.

A. Aggregate Function (no GROUP BY)

Query 1:

1. Find the total number of books in the library (count of rows in Books table) that have more than 300 pages.

Expected: COUNT()



B. Aggregate Function + WHERE + GROUP BY

Query 2:

1. For each publisher, calculate the average number of pages of books they have published, but only for books published after the year 2000 (publication_year > 2000).
2. Show publisher_id and avg_pages.
3. Only include publishers who have at least 2 books meeting the condition.
4. Expected: AVG(), WHERE publication_year > 2000, GROUP BY publisher_id, HAVING COUNT(*) >= 2

```
167 •   select p.publisher_id, ROUND(avg(b.pages), 0) AS avg_pages
168     from publishers p left join books b
169     on p.publisher_id = b.publisher_id
170     and b.publication_year > 2000
171     GROUP BY p.publisher_id
172     HAVING COUNT(*) >= 2;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

publisher_id	avg_pages
4	272

C. Aggregate Function + GROUP BY + HAVING (no WHERE clause filtering before GROUP BY)

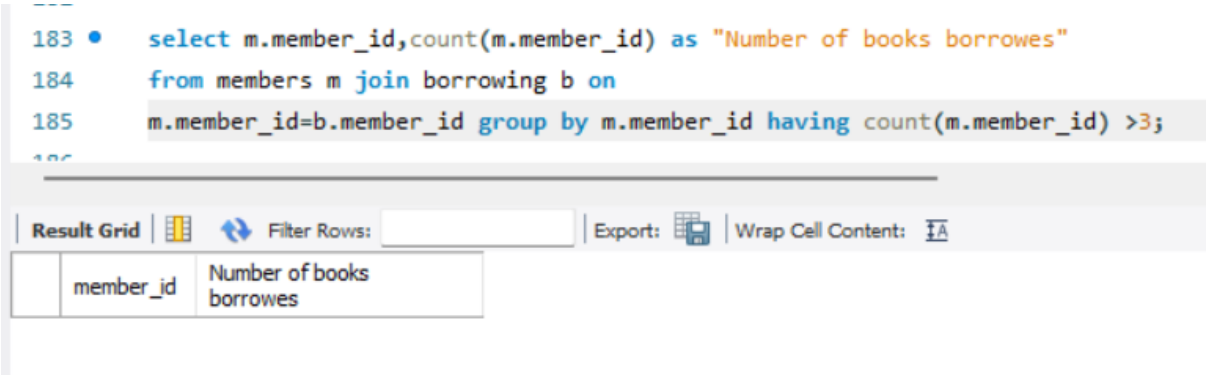
Query 3:

1. Find member IDs who have borrowed more than 3 books in total (counting each borrowing record) across any dates.
2. Show member_id and total_borrowings.
3. Only include members with more than 3 borrowings.

Note: Do not filter by date or any other condition in WHERE (aggregate all borrowings first).

Expected: COUNT(*), GROUP BY member_id, HAVING COUNT(*) > 3

(No WHERE clause before grouping.)



D. Aggregate Function + WHERE + GROUP BY + HAVING (all together)

Query 4:

1. total_books_borrowed (COUNT of borrowings)
2. total_fines_incurred (SUM of fine_amount from Fines table)
3. Only show members whose total fines incurred exceed 15.00 and who borrowed at least 2 books in 2024.

Expected:

1. WHERE (year filter on borrow_date)
2. GROUP BY member_id, full_name
3. SUM(fine_amount) with **COALESCE**(fine_amount, 0)
4. HAVING (two conditions: SUM(...) > 15 AND COUNT(borrow_id) >= 2)

```
191
192 •   select m.member_id,m.full_name,count(b.borrow_id) as total_books_borrowed,
193     sum(coalesce(f.fine_amount, 0)) as total_fines_incurred
194     from members m join borrowing b on m.member_id = b.member_id
195     left join fines f on b.borrow_id = f.borrow_id
196     where year(b.borrow_date) = 2024
197     group by m.member_id, m.full_name
198     having sum(coalesce(f.fine_amount, 0)) > 15
199     and count(b.borrow_id) >= 2;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

member_id	full_name	total_books_borrowed	total_fines_incurred
1	Nischal Shrestha	2	130.00
2	Nirjal Shrestha	2	220.00
3	Priya Poudel	2	300.00